

TRAINING LANGUAGE MODELS TO EXPLAIN THEIR OWN COMPUTATIONS

Parisa Mohammadi

parisainthestars@gmail.com

This report presents our reimplementation and partial reproduction of the experiments in *Training Language Models to Explain Their Own Computations*. Our goal is to examine whether the paper’s main findings can be replicated under a smaller-scale experimental setting. In particular, we reproduce experiments on feature description, activation patching, and input ablation, while also evaluating the claim that language models are better able to explain their own computations than those of a different model family. In Section 4, we further discuss several potential directions for extending this work, motivated by our empirical observations and the limitations encountered during the reproduction study.

1 A SHORT SUMMARY OF THE PAPER

Li et al. investigate whether large language models can be trained to faithfully explain their own internal computations rather than only their final outputs Li et al. (2025). The paper treats existing interpretability methods as a source of supervision and fine-tunes *explainer* models to produce natural-language descriptions for three tasks: (1) what information individual features encode, (2) how intervening on internal activations changes model behavior, and (3) how specific input tokens influence the model’s answer. Trained on only tens of thousands of examples, these explainers generalize to new features, interventions, and inputs. A key finding is that models explain their own computations better than equally or more capable models explain someone else’s computations, suggesting that “*privileged access*” to internal activations is a powerful signal. Overall, the work proposes a scalable way to turn low-level interpretability measurements into human-readable explanations, and argues that explanation should be treated as a trainable capability of language models rather than a purely post-hoc analysis problem. In this project, we aim to reproduce a subset of the paper’s results and critically analyze both its strengths and limitations. Our code is available on GitHub.

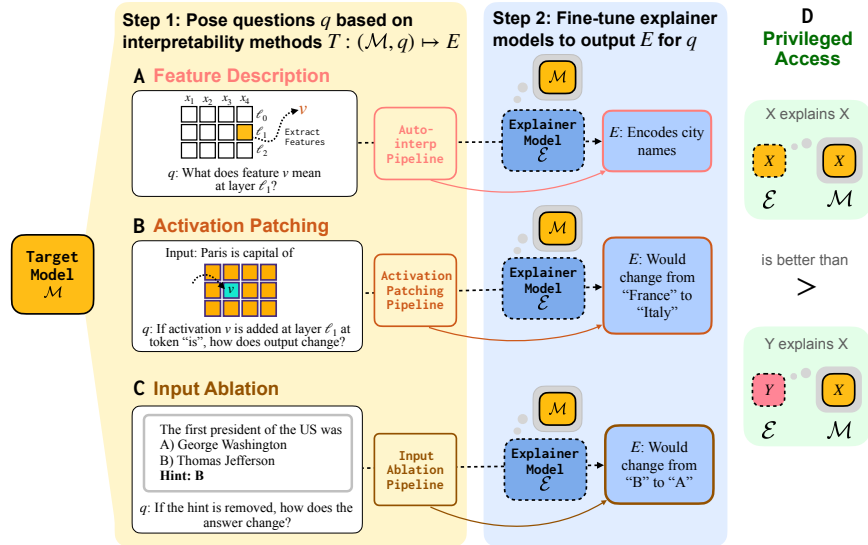


Figure 1: Overview of the method for training models to describe their own internal procedures.

2 EXPERIMENTAL SETUP

In this section, we aim to describe the experimental framework used to reproduce a subset of the results from Li et al. (2025). We organize this section by task, and for each one we summarize (i) the core intuition, (ii) the training objective, and (iii) the specific implementation choices we used in our reproduction.

2.1 TASK 1: FEATURE DESCRIPTION

Intuition. Individual hidden features in transformer models often activate for semantically coherent concepts (e.g., programming keywords, gendered names). The paper trains an explainer model to verbalize what a given feature “means,” based on examples where the feature strongly activates. The expectation is that explainers learn to translate activation patterns into short conceptual summaries.

Training objective. The explainer receives as input a feature direction v from layer ℓ of the target model and is trained to output a short natural-language description E that characterizes when this feature is highly active. Due to the *polysemantic* nature of an LLM’s hidden representations, the direction v is typically taken from a Sparse Auto-Encoder (SAE) dictionary, since these decomposed features tend to isolate more interpretable concepts than raw activations.

The paper assigns to each feature a target description $T_{\text{feat}}(\mathcal{M}, v, \ell)$ by selecting text that best correlates with that feature’s activations. To obtain these targets, the paper trains a small “simulator” language model that predicts how strongly a feature will activate given a candidate explanation E . The final $T_{\text{feat}}(\mathcal{M}, v, \ell)$ is chosen as the explanation whose simulated activations best correlate with the feature’s true activations across data, effectively selecting the E that most accurately predicts where the feature fires.

A limitation is that this pipeline requires training the simulator itself, adding non-trivial computational overhead. The simulator is then reused both to select the best explanation at inference time and to score interpretability quality, which means that evaluation is partly tied to the assumptions and accuracy of the simulator model.¹

During training, the feature vector v is inserted as a continuous token into the explainer’s embedding layer (projected if dimensions differ), and the explainer is optimized with a standard language-modeling loss:

$$\mathcal{L}_{\text{feat}} = -\mathbb{E}_{v, \ell} [\log p_{\theta}(E = T_{\text{feat}}(\mathcal{M}, v, \ell) \mid \Pi_{\ell} v)]. \quad (1)$$

This encourages the model to produce explanations that agree with the automatically derived SAE-based feature descriptions.

Experimental setting.

(i) **Dataset.** Following the original paper, feature directions v are obtained from residual-stream SAEs trained on Llama-3.1-8B using the Llama-Scope framework. In particular, we use the Llama-3.1-8B-LXR-32x dictionary, which maps each layer’s residual stream into roughly 131K sparse features. Ground-truth descriptions E for these features are taken from Neuronpedia² (Lin, 2023), which provides human-annotated summaries for many SAE components. These feature–description pairs are then used to supervise the explainer model during training.

For our experiments, we randomly sample 1K features for training and 50 for testing per layer, resulting in approximately 32K training features overall. We retrieve the corresponding human explanations via the Neuronpedia API. Because each feature requires a separate API call, constructing the dataset took roughly 14 hours, which constrained us from using the full feature set.

(ii) **Models & Training.** We train two explainer models: Qwen/Qwen3-8B and meta-llama/Llama-3.1-8B, each for three epochs. In both cases the target model and explainer share the same hidden dimensionality, so we do not use the linear projection layer described in the paper. During training, for each example we randomly select one of five prompt templates from Appendix D of the original paper. We also perform ablations where explainers are trained on specific subsets of layers and on different dataset sizes; results are reported in Section 3.

(iii) **Evaluation.** Following the paper, we use GPT-4.1-mini as a judge to measure similarity between predicted and ground-truth feature descriptions, using the evaluation template described in Appendix A.2.

2.2 TASK 2: ACTIVATION PATCHING

Intuition. Activation patching asks how the model’s output changes when a specific hidden feature is overwritten using activations from a different input. The explainer is asked to *predict* the downstream effect of the intervention. The hypothesis is that a well-trained explainer can anticipate which internal manipulations actually matter for behavior.

¹The paper also does not specify how many explanation candidates are generated before taking the argmax over E . For these reasons, we discard this component of the pipeline in our reproduction. Instead, we use an LM-judged score for evaluation, and the explainer is allowed to generate only a single explanation at inference time.

²<https://www.neuronpedia.org/>

Training objective. For an original input x and a patched version $\tilde{x}$, the target model produces outputs o and $\tilde{o}$. The explainer receives a description of the intervention and is trained to generate a natural-language account of the effect:

$$\mathcal{L}_{\text{patch}} = -\log p_{\theta}(y \mid \text{patch description}, x),$$

where y encodes whether the answer changes and, when applicable, the predicted outcome after patching.

Experimental setting.

(i) Dataset. We build an activation-patching dataset from the *CounterFact* benchmark (Meng et al., 2023), which pairs factual prompts with counterfactual variants. Each item yields a factual prompt x and a counterfactual prompt x' , both converted into multiple-choice questions by sampling distractors that share the same relation.

For each (x, x') pair we patch activations from x' into x . Model layers are grouped into four blocks; for each block and token position t , we average hidden states

$$v = \frac{1}{|\mathcal{L}|} \sum_{\ell \in \mathcal{L}} h_{\ell,t}(x')$$

and replace $h_{\ell,t}(x)$ accordingly. We label each intervention by whether the model’s next-token prediction changes. Tokens are assigned to semantic categories (subject, relation, answers, distractors, context), and the dataset is balanced across layer blocks and outcomes.

(ii) Models & Training. Activation-patching experiments are conducted on the Qwen/Qwen3-8B model. The model is loaded in half precision with automatic device mapping and evaluated in inference mode. We train an explainer model whose task is to predict the causal effect of an activation patch. Each training example is formatted as a natural-language query describing the intervention, including the input text, token position, and layer block. The intervention vector v is injected directly into the model’s embedding space using a special marker token.

The explainer produces a structured natural-language output indicating either (i) that the output remains unchanged, or (ii) that the output changes to a specific next token. The explainer is trained using a standard negative log-likelihood objective over the target explanation text. Continuous intervention vectors are normalized and inserted at runtime by a custom data collator.

(iii) Evaluation. We evaluate how accurately the explainer predicts the causal effect of activation patching on the base model. Each example is labeled as *changed* or *unchanged* depending on whether the patched forward pass alters the next-token prediction. The explainer must infer this outcome from the intervention description. We report **macro F1** for this task, giving equal weight to both classes and avoiding bias toward the more frequent unchanged cases.

For instances where the model’s output changes, we also check whether the explainer correctly identifies the new next token. We report this as **content-match accuracy**, defined as exact equality with the patched model’s output.

Metrics are further broken down by layer block, token category, and intervention position to analyze how causal effects vary across depth and semantic roles.

2.3 TASK 3: INPUT ABLATION

Intuition. In the input-ablation task, a question is modified by removing a “hint” token. The explainer must predict whether the target model’s answer will change and, if so, what the new answer will be. This task probes whether explainers understand how specific pieces of input influence model predictions.

Training objective. For each input x and ablated subset $\tilde{x}$, we query the target model once on x and once on $x \setminus \tilde{x}$. The explainer receives the multiple-choice question together with the hint tokens $\tilde{x}$ and is trained to predict *two* quantities in a single text output: (i) whether the answer changes when the hint is removed (the *has-changed* label), and (ii) the new answer when it changes (the *content* of the change). Formally, the explainer is optimized to match the automated ablation label $T_{\text{input}}(\mathcal{M}, x, \tilde{x})$:

$$\mathcal{L}_{\text{input}} = -\mathbb{E}_{x, \tilde{x}} [\log p_{\theta}(E = T_{\text{input}}(\mathcal{M}, x, \tilde{x}) \mid x, \tilde{x})].$$

The explanation is generated as a single text sequence, and we apply masked loss so that gradients only flow through the explainer’s generated tokens.

Experimental setting.

(i) Dataset. We construct our own input-ablation dataset from the MMLU benchmark, using meta-llama/Llama-3.1-8B as the target model. We sample 14K multiple-choice questions from MMLU³

³<https://huggingface.co/datasets/cais/mmlu>

for training. For each question, the target model answers the original prompt and the hint-ablated version, and the explainer is trained to verbalize whether the answer changes and, if so, what the new answer becomes. Some representative samples of the processed training data are provided in Appendix A.

(ii) *Models & Training.* We train two separate explainer: (i) the same LLaMA model and (ii) Qwen/Qwen3-8B. Explainers are trained for three epochs using the input-ablation prompt templates from Appendix D of the paper.

(iii) *Evaluation.* Evaluation follows the paper’s protocol, including the conditional definition of content accuracy (i.e., content is only evaluated on examples where the model’s prediction changes).

3 RESULTS

In this section we report the empirical performance of our reproduced components. Our goal is not to fully match the original paper’s scale, but to evaluate whether the key qualitative claims hold under our more limited reimplementaion.

3.1 FEATURE DESCRIPTION EXPLAINERS

We first evaluate explainer models trained to verbalize SAE feature meanings. Both LLaMA-3.1-8B and Qwen-3-8B are trained to predict natural-language descriptions of features, given the target layer and feature vector as input. Performance is measured using the LM-Judge score between predicted and ground-truth Neuronpedia descriptions.

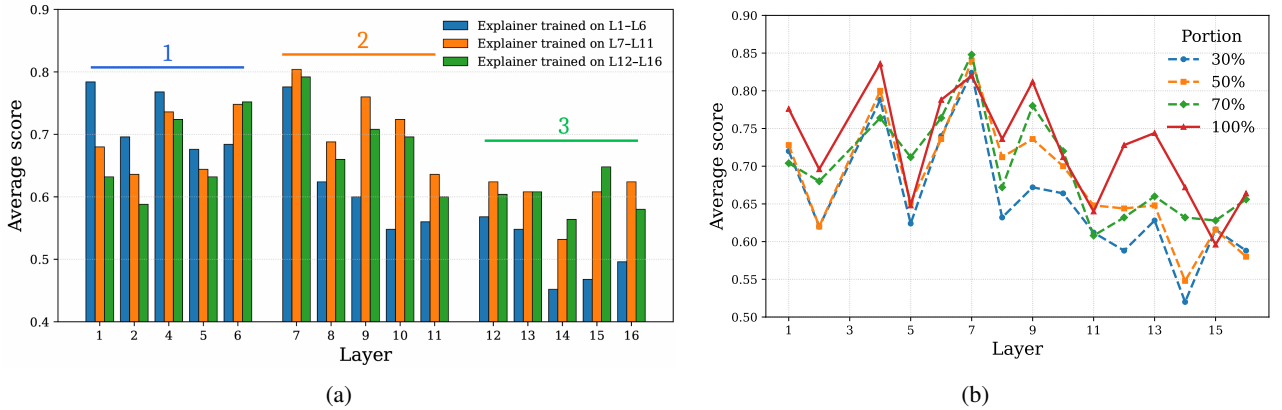


Figure 2: **Feature-description performance under different training conditions.** (a) Layer-restricted explainer trained on different subsets of layers evaluated across all layers. Each explainer performs best on the layer range it was trained on, indicating limited cross-layer generalization. (b) Effect of training-data size. Models trained on 30–70% of the data perform only slightly worse than the full 100% model, suggesting that feature description is relatively sample-efficient.

In Figure 2(a) we train three separate explainer, each on a different subset of layers: layers 1–6, 7–11, and 12–16 of the LLaMA residual stream. We then evaluate every explainer on all layers from 1 to 16. Each bar group therefore shows how well a given explainer generalizes across depth. The results indicate that each explainer performs best on same layer range it was trained on (e.g., explainer trained on layers 1–6 achieves the highest scores in [region 1](#)). This also suggests that the models trained on earlier layers do not transfer perfectly across the network, specifically deeper layers.

Figure 2(b) compares models trained on different fractions of the available training set (30%, 50%, 70%, and 100%). For the smaller fractions, we randomly downsample the training features. As expected, the model trained on the full dataset (100%) generally achieves the best performance, but the gap to the 70% and even 50% models is relatively modest. This supports the paper’s claim that the feature-description paradigm is sample-efficient: reasonable explanation quality can be achieved with far fewer examples than the full corpus.

Finally, Figure 3 compares two explainer backbones, meta-llama/Llama-3.1-8B and Qwen/Qwen3-8B, when explaining features of the same target model (LLaMA). Two main observations emerge:

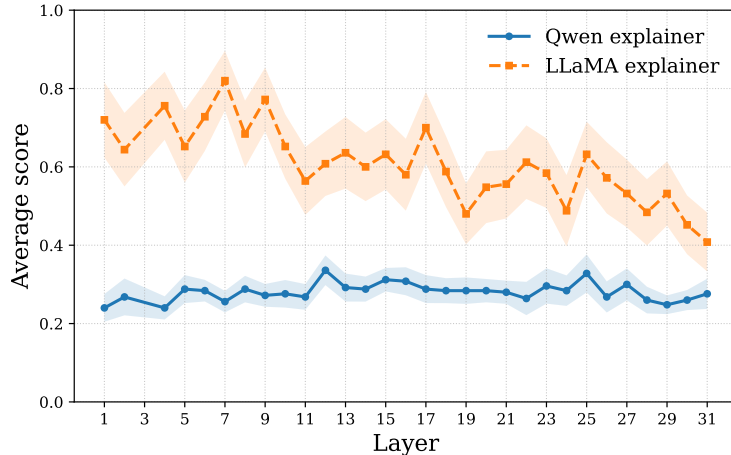


Figure 3: **Comparison of explainer backbones on LLaMA features.** LLaMA- and Qwen-based explainers are trained to describe SAE features from the same target model. The LLaMA explainer consistently performs better, supporting the “privileged access” hypothesis. Performance is also higher in earlier layers, suggesting that deeper representations are harder to interpret.

1. The results support the paper’s “privileged access” hypothesis: the LLaMA-based explainer, which shares architecture and pretraining with the target model, consistently outperforms the Qwen-based explainer on this task. This suggests that models are better at explaining their own internal features than a different model family.
2. Explanation quality tends to be higher in earlier layers and degrades in deeper layers. Intuitively, early features capture relatively local and disentangled patterns, whereas later layers combine many signals in a highly non-linear way, making them harder to interpret. This highlights a limitation of the current approach: it is more effective for shallow or mid-level representations than for deeply entangled ones.

Some qualitative examples are shown in Figure 4, illustrating cases where the explainer correctly captures the underlying feature semantics. In contrast, explainer models without training fail entirely on this task and produce meaningless outputs.

3.2 ACTIVATION PATCHING

Table 1 shows that removing any single component of the intervention description causes only a modest drop in performance. The full model performs best, but each ablated variant still retains substantial predictive ability, indicating that the explainer draws on multiple sources of causal information rather than any single cue.

Target	Has-Changed F1 (%)	Content Match (%)
Explainer (full)	65.5 ± 0.7	50.2 ± 0.4
Activation only	64.2 ± 0.9	51.8 ± 0.5
Layer only	64.1 ± 0.5	51.1 ± 0.3
Token only	63.9 ± 0.6	52.4 ± 0.1

Table 1: Evaluation results of the explainer model across different intervention targets.

3.3 INPUT ABLATION EXPLAINERS

We now evaluate the ability of explainers to predict the effect of removing hint tokens from multiple-choice questions. In all experiments, the target model is meta-llama/Meta-Llama-3.1-8B-Instruct, and the explainer is either the same LLaMA model or Qwen/Qwen3-8B. The first two rows in Table 2 correspond to explainers trained using the proposed framework, while the last two rows show the raw, untrained models.

The results show a clear separation between trained and untrained models. The trained LLaMA explainer achieves 68.2% exact match and 74.8% F1 on predicting whether the answer changes, while the trained Qwen explainer per-

Example 1 Layer 1 — Feature 66299 Predicted: phrases related to celebration or acknowledgment of achievements Ground truth: phrases indicating celebration or recognition of occasions	Score = 5
Example 2 Layer 5 — Feature 59655 Predicted: terms related to clothing and garment construction Ground truth: words and phrases associated with fashion and style details	Score = 5
Example 3 Layer 5 — Feature 66117 Predicted: references to corporations and their activities Ground truth: terms related to the semiconductor industry and corporate dominance	Score = 4
Example 4 Layer 15 — Feature 28224 Predicted: references to notable individuals and their achievements Ground truth: names of notable individuals or figures	Score = 5
Example 5 Layer 1 — Feature 80220 Predicted: phrases related to probability or likelihood Ground truth: phrases related to probabilities and chances of events occurring	Score = 5
Example 6 Layer 31 — Feature 30161 Predicted: words related to the concept of abundance and sufficiency Ground truth: symbols and characters from different languages and scripts	Score = 1
Example 7 Layer 1 — Feature 38976 Predicted: mathematical symbols and operations Ground truth: mathematical or programming symbols and operators	Score = 5

Figure 4: Qualitative examples where the explainer model (LLaMA) closely matches the human explanation of SAE features. Higher scores indicate closer semantic agreement.

forms somewhat worse but still far above chance. In contrast, the untrained models produce almost no correct structured explanations (0% exact match) and essentially fail to detect change. This strongly suggests that fine-tuning is *necessary* for models to gain interpretive competence; pretraining alone does not yield usable ablation explanations.

A second pattern echoes the “privileged access” hypothesis from the paper: the trained LLaMA explainer (which shares architecture and training with the target model) substantially outperforms the trained Qwen explainer on every metric. This supports the idea that models are better at reasoning about their own internal behavior than about the computations of another model family.

Explainer	Exact Match	Has-Changed F1	Content Match
LLaMA (trained)	68.23%	74.77%	81.44%
Qwen (trained)	49.35%	55.03%	57.35%
LLaMA (untrained)	0.00%	1.86%	50.00%
Qwen (untrained)	0.00%	3.66%	16.67%

Table 2: Input-ablation explainer performance on the validation split (620 samples). Trained explainers significantly outperform raw models, and the LLaMA explainer outperforms Qwen, supporting the privileged-access hypothesis.

4 LIMITATIONS AND NEXT STEPS

A key limitation is that explanations work best in early layers but deteriorate in deeper ones (see Fig. 3). Early layers encode relatively simple, local patterns, while later layers mix many signals and implement more abstract reasoning, making them harder to describe. Thus, the method is most reliable for shallow representations and weakest where behavior becomes complex. A natural next step is to develop explainers that summarize interactions across layers or circuits, rather than focusing on isolated features.

The favorite limitation of almost all reviewers: “explaining your own computations” is studied in a relatively narrow setting. Privileged access is instantiated as access to internal activations of a single base model, and success is measured on three tasks that are all short-context. It is unclear how well this paradigm extends to more complex behaviors, such as long-horizon reasoning, planning, or multi-step dialogue, where explanations may need to reference evolving internal states and cross-token dependencies rather than static features or single-token changes.

Personally, I found Task 1 (feature description) to be the most interesting part of the paper. A natural question is whether this framework can be extended to the safety domain, where threat models are often described verbally. For instance, could it help interpret the underlying concept behind an adversarial trigger (backdoor)? If so, would the resulting description align with the attacker’s intention? In that case, we would effectively obtain an *interpretable attack*. Such explanations could then serve as additional signals or supervision for either strengthening the attack or improving model safety and robustness.

Finally, an interesting direction is to extend this idea from interpreting single latents to interpreting *differences* in latent space. In particular, can intervention or steering directions themselves be explained? If so, this could make comparison-based alignment techniques such as DPO (Rafailov et al., 2024), IPO (Azar et al., 2023), and steering-vector methods more transparent, by revealing the behavioral constraint each direction enforces. I have already expanded my research proposal to include this idea, and in Section 3.4 I argue that interpreting latent *differences* may be as important as interpreting latent *features*.

5 CONCLUSION

Our results show that explainers can learn meaningful, causal interpretations of model behavior when trained with targeted supervision. Fine-tuning is essential while model-matched explainers benefit from privileged access to their own representations.

REFERENCES

- Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences, 2023. URL <https://arxiv.org/abs/2310.12036>.
- Belinda Z. Li, Zifan Carl Guo, Vincent Huang, Jacob Steinhardt, and Jacob Andreas. Training language models to explain their own computations, 2025. URL <https://arxiv.org/abs/2511.08579>.
- Johnny Lin. Neuronpedia: Interactive reference and tooling for analyzing neural networks. <https://www.neuronpedia.org>, 2023. Software available at [neuronpedia.org](https://www.neuronpedia.org).
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt, 2023. URL <https://arxiv.org/abs/2202.05262>.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. URL <https://arxiv.org/abs/2305.18290>.

APPENDIX

A INPUT-ABLATION DATASET EXAMPLES

Below we show two complete examples from our constructed dataset, including all fields used during training.

```
{
  "subject": "abstract_algebra",
  "question": "Statement 1 | Every group of order 42 has a normal subgroup of order 7. Statement 2 | Every group of order 42 has a normal subgroup of order 8.",
  "choices": ["True, True", "False, False", "True, False", "False, True"],
  "correct_answer": 2,
  "correct_letter": "C",
  "target_model": "meta-llama/Meta-Llama-3.1-8B-Instruct",
  "hint_letter": "D",
  "hint_text": "Hint: D",
  "prompt_target_nohint": "Statement 1 | Every group of order 42 has a normal subgroup of order 7. Statement 2 | Every group of order 42 has a normal subgroup of order 8.\n\nA. True, True\nB. False, False\nC. True, False\nD. False, True\n\nAnswer:",
  "prompt_target_with_hint": "Statement 1 | Every group of order 42 has a normal subgroup of order 7. Statement 2 | Every group of order 42 has a normal subgroup of order 8.\n\nA. True, True\nB. False, False\nC. True, False\nD. False, True\n\nHint: D\n\nAnswer:",
  "target_answer_nohint": "C",
  "target_answer_with_hint": "D",
  "has_changed": 1,
  "explainer_prompt": "[SYSTEM]\nThe following are multiple choice questions (with a correct answer). Output only the answer letter (A, B, C, or D) and nothing else, in the format Answer: x, where x is one of A, B, C, or D.\n\n[USER]\nQuestion:\nStatement 1 | Every group of order 42 has a normal subgroup of order 7. Statement 2 | Every group of order 42 has a normal subgroup of order 8.\n\nA. True, True\nB. False, False\nC. True, False\nD. False, True\n\nHint: D\n\nIf the hint were removed how would the assistant answer change?\n\n[ASSISTANT]\n",
  "explainer_output": "The most likely output would change to <<<Answer: C>>>."
}

{
  "subject": "abstract_algebra",
  "question": "Statement 1 | If  $T: V \rightarrow W$  is a linear transformation and  $\dim(V) < \dim(W) < 1$ , then  $T$  must be injective. Statement 2 | Let  $\dim(V) = n$  and suppose that  $T: V \rightarrow V$  is linear. If  $T$  is injective, then it is a bijection.",
  "choices": ["True, True", "False, False", "True, False", "False, True"],
  "correct_answer": 0,
  "correct_letter": "A",
  "target_model": "meta-llama/Meta-Llama-3.1-8B-Instruct",
  "hint_letter": "D",
  "hint_text": "Hint: D",
  "prompt_target_nohint": "Statement 1 | If  $T: V \rightarrow W$  is a linear transformation and  $\dim(V) < \dim(W) < 1$ , then  $T$  must be injective. Statement 2 | Let  $\dim(V) = n$  and suppose that  $T: V \rightarrow V$  is linear. If  $T$  is injective, then it is a bijection.\n\nA. True, True\nB. False, False\nC. True, False\nD. False, True\n\nAnswer:",
  "prompt_target_with_hint": "Statement 1 | If  $T: V \rightarrow W$  is a linear transformation and  $\dim(V) < \dim(W) < 1$ , then  $T$  must be injective. Statement 2 | Let  $\dim(V) = n$  and suppose that  $T: V \rightarrow V$  is linear. If  $T$  is injective, then it is a bijection.\n\nA. True, True\nB. False, False\nC. True, False\nD. False, True\n\nHint: D\n\nAnswer:",
  "target_answer_nohint": "D",
  "target_answer_with_hint": "D",
  "has_changed": 0,
  "explainer_prompt": "[SYSTEM]\nThe following are multiple choice questions (with a correct answer). Output only the answer letter (A, B, C, or D) and nothing else, in the format Answer: x, where x is one of A, B, C, or D.\n\n[USER]\nQuestion:\nStatement 1 | If  $T: V \rightarrow W$  is a linear transformation and  $\dim(V) < \dim(W) < 1$ , then  $T$  must be injective. Statement 2 | Let  $\dim(V) = n$  and suppose that  $T: V \rightarrow V$  is linear. If  $T$  is injective, then it is a bijection.\n\nA. True, True\nB. False, False\nC. True, False\nD. False, True\n\nHint: D\n\nIf the hint were removed how would the assistant answer change?\n\n[ASSISTANT]\n",
  "explainer_output": "The output would remain unchanged from <<<Answer: D>>>."
}
```

Figure 5: Two full JSON samples from the input-ablation dataset. The explainer is trained to produce the explainer_output field.